

The KMP Algorithm

- Introduction to Knuth-Morris-Pratt
- Goal: Find a pattern P (length m) in Text T (length n) efficiently.
- Applications: DNA sequencing, Search Engines, Data Mining.

The Naive (Brute Force) Approach

- How it works:
 - - Slide the pattern one character at a time.
 - - If a mismatch occurs, shift right by 1.
- Complexity:
 - - Worst Case: $O(m * n)$
 - - Inefficient because it re-evaluates characters we already matched.

The Naive (Brute Force) Approach

- for i from 0 to n - m:
- for j from 0 to m - 1:
- if Text[i + j] != Pattern[j]:
- break // Mismatch found
- if j == m:
- return i // Pattern found at index i

Why is it inefficient?

- **Worst Case Example:**
- **Text:** AAAAAAAAAAAB
- **Pattern:** AAAB
- **Problem:** When a mismatch occurs, we throw away all the information we gained from the partial match and only shift by **one** step. We re-examine characters we already know match!

The KMP Insight

- Key Idea:
- - When a mismatch happens, we have already matched a prefix.
- - We can use this knowledge to shift MORE than 1 step.
- - We never move the Text pointer backward.

Visual Intuition:

- If we matched "ABABA" and failed at the next character, we know the text contains "ABABA" at that spot. We don't need to check the text again; we just need to align the pattern smartly.

The LPS Array

- LPS = Longest Proper Prefix which is also a Suffix
- It tells us where to reset the pattern pointer (j).
- Example for 'ABA':
 - - Prefix: A
 - - Suffix: A
 - - LPS Value: 1

Calculating the LPS Array (Example)

- Let's calculate LPS for the Pattern: **A B A B A C**

Index (i)	Sub-pattern (P[0...i])	Longest Proper Prefix which is also a Suffix	LPS Value
0	A	"" (None)	0
1	A B	"" (None)	0
2	A B A	"A"	1
3	A B A B	"AB"	2
4	A B A B A	"ABA"	3
5	A B A B A C	"" (None, C breaks the pattern)	0

The KMP Search Algorithm

The Logic:

1. Initialize pointers: i for Text, j for Pattern.
2. Compare Text[i] and Pattern[j]
3. **If Match:** Increment both i and j
4. **If Mismatch (after some matches):**
 - Do **NOT** move i backwards (unlike Brute Force).
 - Set $j = \text{LPS}[j-1]$. (Jump to the last matching prefix).
5. **If Mismatch (at the start, $j=0$):**
 - Just increment i

Walkthrough Example - Step 1

- **Text:** A B A B A B A C **Pattern:** A B A B A C **LPS:**
[0, 0, 1, 2, 3, 0]
- Comparison
- $i = 0, 1, 2, 3, 4$
- Text: A B A B A B A C ...
- Pattern: A B A B A C
- Matches: ✓ ✓ ✓ ✓ ✓ ✗

Text: A B A B A B A C **Pattern:** A B A B A C

LPS: [0, 0, 1, 2, 3, 0]

- We match 5 characters (A B A B A).
- Mismatch at index 5 (B vs C).
- Action: Look at $LPS[5-1] = LPS[4] = 3$
- We shift the pattern so index 3 (B) aligns with the mismatch.

Walkthrough Example - Step 2

Current State: i stays at 5 j moves to 3

- $i = 5$
- Text: A B A B A B A C ...
- Pattern: A B A B A C
- Matches: ✓

Walkthrough Example - Step 2 cont..

The pattern shifted right.

We know A B A (prefix) matches the A B A (suffix)
we just saw.

Now compare Text[5] ('B') with Pattern[3] ('B'). Match
Increment i and j.

Walkthrough Example - Step 3

- $i = 6, 7$
- Text: ... A B A B A B A C
- Pattern: ... A B A B A C
- Matches: ✓ ✓ ✓

Text: A B A B A B A C ...

Pattern: A B A B A C

- Compare remaining characters.
 - Text[6] ('A') matches Pattern[4] ('A').
 - Text[7] ('C') matches Pattern[5] ('C').
 - Result: Pattern found at index $i - j = 7 - 5 = 2$

Complexity Analysis

- Time Complexity:
 - - Preprocessing: $O(m)$
 - - Searching: $O(n)$
 - - Total: $O(n + m)$
- Space Complexity:
 - - $O(m)$ for the LPS table.